

KONGU ENGINEERING COLLEGE

(AUTONOMOUS)

PERUNDURAI, ERODE – 638 060

EXPERIMENT - 3

PROJECT TITLE

**AIR POLLUTION MONITORING SYSTEM USING MQ2 GAS
SENSOR**

Submitted by

ABISHEK R S - 24MER002

GOWRISANKAR K - 24MER010

NAVEENRAJ K - 24MER035

PRAVEEN N M - 24MER043

AIR POLLUTION MONITORING SYSTEM USING MQ2 GAS SENSOR

1. Project review :

The Air Pollution Monitoring System Using MQ2 Gas Sensor is a compact prototype designed to detect harmful gases and monitor the level of air pollution in the surrounding environment. The system uses an ESP8266 NodeMCU as the central controller and an MQ2 gas sensor to detect the presence of smoke and combustible gases.

The MQ2 sensor continuously senses the gas concentration in the surrounding air and sends the sensor readings to the ESP8266 NodeMCU. The controller processes these values and compares them with a predefined safety limit. When the gas level exceeds the selected limit, a buzzer provides an audible warning, and an LED or relay module can be activated to indicate an unsafe air condition.

The proposed system provides a simple, low-cost, and automatic method of monitoring air pollution and harmful gas leakage. It can also be further developed with Wi-Fi connectivity, cloud monitoring, mobile notifications, data logging, and IoT-based air quality monitoring.

2. Problem Statement :

Air pollution and gas leakage can create unsafe environmental conditions in homes, industries, laboratories, kitchens, and other public places. Harmful gases and smoke may not always be detected immediately by humans, which can lead to health problems, fire hazards, or other dangerous situations.

Manual monitoring of air quality and gas concentration is inconvenient and does not provide an immediate warning when the pollution level becomes unsafe. Therefore, there is a need for a simple and low-cost system that can automatically detect harmful gases and provide an alert.

The proposed system addresses these problems by integrating an MQ2 gas sensor, ESP8266 NodeMCU, buzzer, LED, and relay module. The MQ2 sensor detects smoke and combustible gases, while the ESP8266 processes the sensor readings. When the gas level exceeds the predefined limit, the buzzer provides an alert and the LED or relay can be activated for warning or ventilation control.

3. Proposed Solution :

The proposed solution consists of an ESP8266-based Air Pollution Monitoring System. The MQ2 gas sensor is used to detect smoke and combustible gases in the surrounding environment and sends the sensor readings to the ESP8266 NodeMCU.

The ESP8266 processes the sensor readings and compares the measured gas level with a predefined safety limit. If the gas level exceeds the selected limit, the ESP8266 activates the buzzer to provide an audible warning. An LED can also indicate the pollution condition, and a relay module can be activated to control a suitable exhaust fan or ventilation system.

Proposed Operation:

- 1 Start the system and initialize the ESP8266.
- 2 Initialize the MQ2 gas sensor, buzzer, LED, and relay.
- 3 The MQ2 sensor detects smoke and harmful gases.
- 4 The MQ2 sensor sends the gas sensor data to the ESP8266.
- 5 ESP8266 processes the gas sensor readings.
- 6 The gas level is compared with the predefined safety limit.
- 7 If the gas level is above the limit, the buzzer is activated.
- 8 The LED provides a visual warning indication.
- 9 The relay can be activated for suitable ventilation or exhaust control.
- 10 If the gas level is normal, the buzzer remains OFF.
- 11 The system continuously repeats the monitoring process.

4. System Architecture :

The Air Pollution Monitoring System consists of five main sections:

Input Section:

The input section consists of:

- . MQ2 Gas Sensor – Detects smoke and combustible gases in the surrounding environment.**

- . The MQ2 sensor sends the measured gas level to the ESP8266 NodeMCU.

Processing Section:

The **ESP8266 NodeMCU** acts as the central controller.

- Receives gas sensor data from the MQ2 sensor.
- Processes the sensor readings.
- Compares the gas level with the predefined safety limit.
- Controls the buzzer, LED, and relay according to the programmed condition.

Actuation Section:

The **Relay Module** is used for cooling control.

- Relay receives a control signal from the ESP8266.

It can control a suitable exhaust fan or other safety device when the gas level becomes high.

Indication Section:

The **Buzzer** and LED provide warning indications.

- ? **Buzzer OFF and LED OFF → Normal air condition.**
- ? **Buzzer ON and LED ON → Gas level exceeds the predefined safety limit..**

Connectivity Section:

The **ESP8266** has built-in Wi-Fi capability.

The system can be further developed for:

- IoT-based air quality monitoring.
- Cloud data storage.
- Mobile notifications.
- Remote monitoring through Wi-Fi.

5. CIRCUIT TABLE :

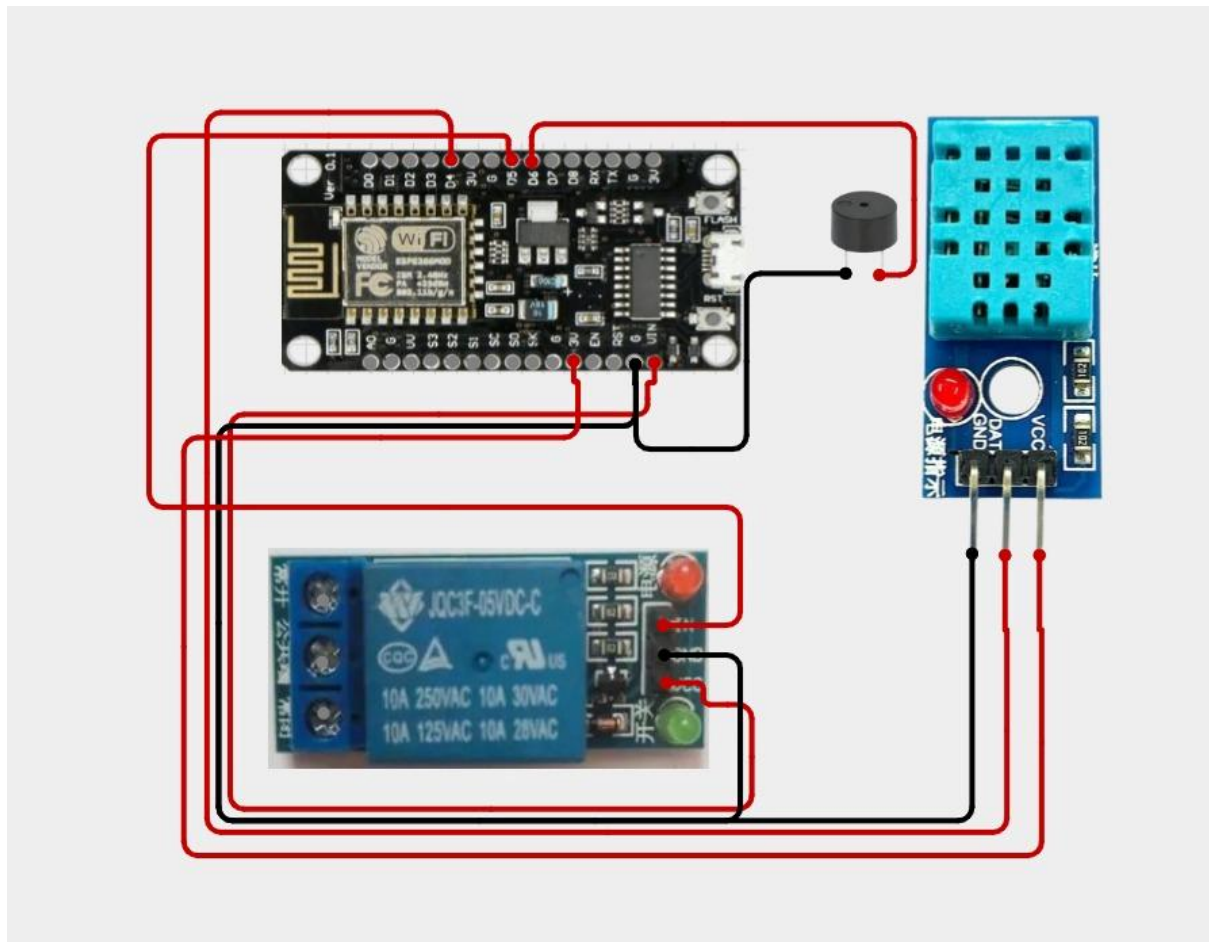
Important Pin Mapping:

ESP8266 NodeMCU Pin Component

GPIO 2	D4 MQ2 Gas Sensor → AO
GPIO 14	D5 Relay Module → IN
GPIO 12	D6 Buzzer → Positive Terminal
GPIO 13	D7 LED → Anode through resistor
5V / VIN	– MQ2 Sensor → VCC
GND	– MQ2 Sensor → GND and Common Ground

6. Circuit Design :

Parameter	Expected Performance
Gas Monitoring	Continuous
Smoke Detection	Continuous
Sensor Response	Real-time Reading
High Gas Level Alert	Buzzer ON
Visual Warning	LED ON
Ventilation Control	Relay ON
Normal Condition	Buzzer OFF and LED OFF



8.Coding

```
// AIR POLLUTION MONITORING SYSTEM
```

```
// ESP8266 NodeMCU + MQ-2 + Buzzer + LED + WiFi
```

```
// + ThingSpeak
```

```
// =====
```

```
#include <ESP8266WiFi.h>
```

```
#include <ESP8266HTTPClient.h>
```

```
// =====
```

```
// WIFI CONNECTION
```

```
// =====
```

```
const char* WIFI_SSID = "praveen";
```

```
const char* WIFI_PASSWORD = "praveen12345";
```

```
// =====
```

```
// THINGSPEAK API
```

```
// =====
```

```
const char* THINGSPEAK_API_KEY = "0IP4GOX0SMNK3JBV";
```

```
const char* THINGSPEAK_SERVER = "http://api.thingspeak.com/update";
```

```
// =====
```

```
// MQ-2 Analog Output-> A0
```

```
// =====
```

```
#define MQ2_PIN A0
```

```
// Buzzer +-> D6 / GPIO12
```

```
#define BUZZER_PIN D6
```

```
// LED Anode-> D5 / GPIO14 through 220 ohm resistor
```

```
#define LED_PIN D5
```

```
// Gas pollution threshold
```

```
// Adjust this value after testing your MQ-2 sensor
```

```
#define GAS_THRESHOLD 200
```

```
// ThingSpeak upload timer
```

```
unsigned long lastThingSpeakUpdate = 0;
```

```
const unsigned long THINGSPEAK_INTERVAL = 15000;
```

```
// =====
```

```
// SETUP
```

```
// =====
```

```
void setup()
```

```
{
```

```
  Serial.begin(9600);
```

```
  pinMode(MQ2_PIN, INPUT);
```



```
pinMode(BUZZER_PIN, OUTPUT);
pinMode(LED_PIN, OUTPUT);

digitalWrite(BUZZER_PIN, LOW);
digitalWrite(LED_PIN, LOW);

// =====
// WIFI CONNECTION
// =====

Serial.println();
Serial.println("=====");
Serial.println(" AIR POLLUTION MONITORING SYSTEM");
Serial.println("=====");

Serial.println("Connecting to WiFi...");

WiFi.begin(WIFI_SSID, WIFI_PASSWORD);

while (WiFi.status() != WL_CONNECTED)
{
    delay(500);
    Serial.print(".");
}

Serial.println();
```

```
Serial.println("WiFi Connected!");
```

```
Serial.print("IP Address: ");
```

```
Serial.println(WiFi.localIP());
```

```
// =====
```

```
// ORIGINAL MQ-2 INITIALIZATION
```

```
// =====
```

```
Serial.println("MQ-2 Sensor Initializing...");
```

```
delay(2000);
```

```
Serial.println("System Ready");
```

```
Serial.println("-----");
```

```
}
```

```
// =====
```

```
// LOOP
```

```
// =====
```

```
void loop()
```

```
{
```

```
  // Read MQ-2 analog value
```

```
  int gasValue = analogRead(MQ2_PIN);
```

```
Serial.print("MQ-2 Gas Level: ");
Serial.println(gasValue);

// =====

// ORIGINAL POLLUTION CHECK

// =====

if (gasValue >= GAS_THRESHOLD)
{
    digitalWrite(LED_PIN, HIGH);
    digitalWrite(BUZZER_PIN, HIGH);

    Serial.println("WARNING: HIGH GAS LEVEL!");
    Serial.println("Air Quality: POOR");
}
else
{
    digitalWrite(LED_PIN, LOW);
    digitalWrite(BUZZER_PIN, LOW);

    Serial.println("Air Quality: NORMAL");
}

Serial.println("-----");

// =====
```

```
// THINGSPEAK DATA UPLOAD

// =====

if (millis()- lastThingSpeakUpdate >= THINGSPEAK_INTERVAL)
{
    lastThingSpeakUpdate = millis();

    if (WiFi.status() == WL_CONNECTED)
    {
        WiFiClient client;
        HTTPClient http;

        String url = String(THINGSPEAK_SERVER);
        url += "?api_key=";
        url += THINGSPEAK_API_KEY;
        url += "&field1=";
        url += String(gasValue);

        Serial.println("Sending data to ThingSpeak...");

        http.begin(client, url);

        int httpCode = http.GET();

        if (httpCode > 0)
        {
```

```
Serial.print("ThingSpeak Response: ");
Serial.println(httpCode);

if (httpCode == 200)
{
    Serial.println("ThingSpeak Update Successful!");
}
else
{
    Serial.println("ThingSpeak Update Failed!");
}
}
else
{
    Serial.print("ThingSpeak Error: ");
    Serial.println(http.errorToString(httpCode));
}

http.end();
}
else
{
    Serial.println("WiFi Disconnected!");
}

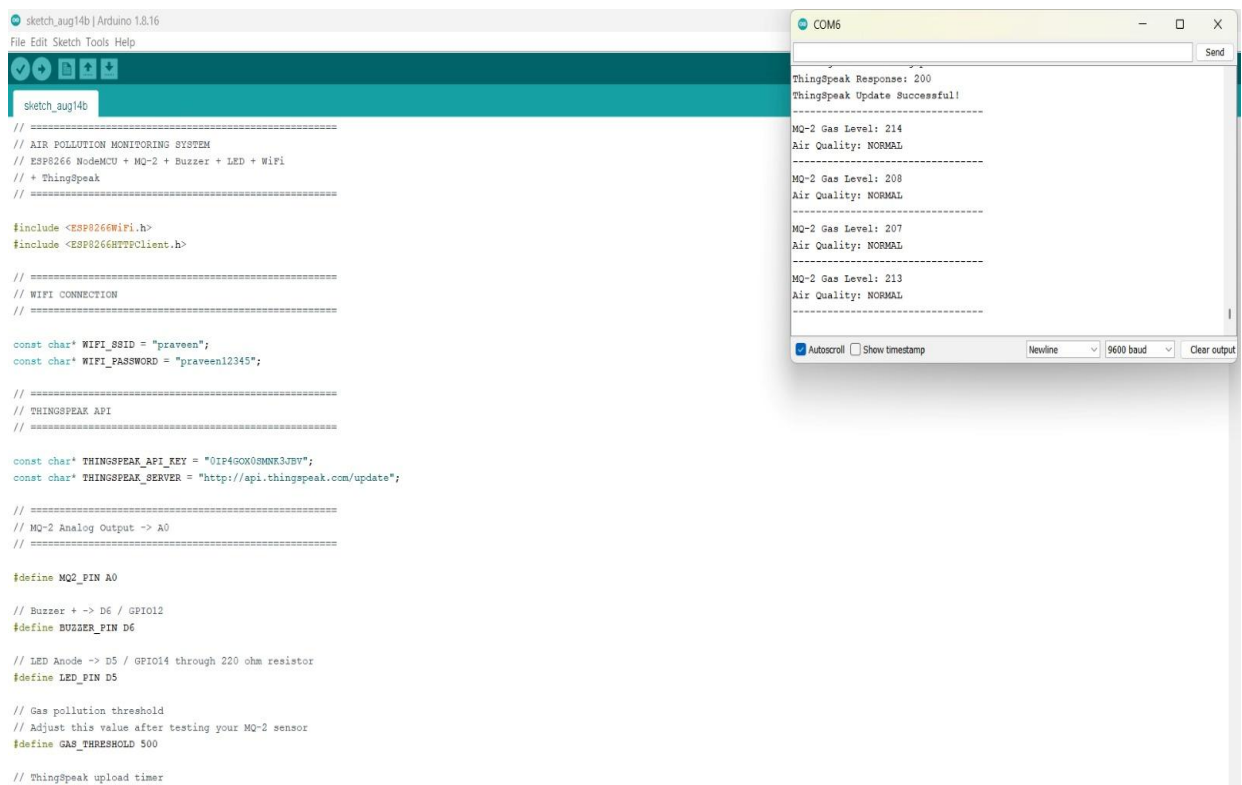
Serial.println("-----");
```

```
}
```

```
delay(1000);
```

```
}
```

CODING OUTPUT:



The screenshot displays the Arduino IDE interface with a sketch named 'sketch_aug14b' and a serial monitor window titled 'COM6'.

Arduino IDE Code:

```
sketch_aug14b
// =====
// AIR POLLUTION MONITORING SYSTEM
// ESP8266 NodeMCU + MQ-2 + Buzzer + LED + Wifi
// + ThingSpeak
// =====

#include <ESP8266Wifi.h>
#include <ESP8266HTTPClient.h>

// =====
// WIFI CONNECTION
// =====

const char* WIFI_SSID = "praveen";
const char* WIFI_PASSWORD = "praveen12345";

// =====
// THINGSPEAK API
// =====

const char* THINGSPEAK_API_KEY = "01P4G0X0SMNK3JBV";
const char* THINGSPEAK_SERVER = "http://api.thingspeak.com/update";

// =====
// MQ-2 Analog Output -> A0
// =====

#define MQ2_PIN A0

// Buzzer + -> D6 / GPIO12
#define BUZZER_PIN D6

// LED Anode -> D5 / GPIO14 through 220 ohm resistor
#define LED_PIN D5

// Gas pollution threshold
// Adjust this value after testing your MQ-2 sensor
#define GAS_THRESHOLD 500

// ThingSpeak upload timer
```

Serial Monitor Output (COM6):

```
Thingspeak Response: 200
Thingspeak Update Successful!

MQ-2 Gas Level: 214
Air Quality: NORMAL
-----
MQ-2 Gas Level: 208
Air Quality: NORMAL
-----
MQ-2 Gas Level: 207
Air Quality: NORMAL
-----
MQ-2 Gas Level: 213
Air Quality: NORMAL
```

9. System Working :

- **Power ON the System**

- The ESP8266 NodeMCU is powered ON.

- **Initialize Components**

- The MQ2 gas sensor, relay module, buzzer, and LED are initialized.

- **Detect Gas and Smoke**

- The MQ2 gas sensor continuously detects the presence of smoke and combustible gases in the surrounding environment.

- **Send Sensor Data**

The MQ2 sensor sends the gas sensor reading to the ESP8266 NodeMCU.

- **Process the Data**

- The ESP8266 receives and processes the sensor readings.

- **Check Gas Level**

The measured gas level is compared with the predefined safety limit.

- **High Temperature Condition**

- If the temperature is above the limit:
 - **Buzzer → ON**
 - **LED → ON**
 - **Relay → ON**
 - Ventilation or exhaust control is activated.

- **Normal Air Condition**

- If the temperature is within the limit:
 - **Buzzer → OFF**
 - **LED → OFF**
 - Relay remains in its normal state.

- **Repeat the Process**

- The MQ2 sensor continuously takes new readings, and the ESP8266 continuously monitors the air condition.

- **10. PROTOTYPE IMPLEMENTATION :**

The prototype is implemented using an ESP8266 NodeMCU, MQ2 gas sensor, relay module, buzzer, LED, breadboard, and jumper wires. The MQ2 gas sensor is connected to the ESP8266 to monitor the presence of smoke and combustible gases in the surrounding environment.

Hardware Setup

1. Connect the MQ2 sensor output → Analog input of the ESP8266 NodeMCU.
2. Connect the Relay IN pin → D5 (GPIO14).
3. Connect the Buzzer → D6 (GPIO12).
4. Connect the LED → D7 (GPIO13) through a suitable resistor.
5. Connect the MQ2 sensor VCC → suitable power supply.
6. Connect the MQ2 sensor GND → GND.
7. Connect all required low-voltage components using jumper wires.
8. Upload the program to the ESP8266 using the Arduino IDE.
9. Power ON the prototype and observe the gas sensor readings.
10. When the gas level exceeds the predefined limit, the buzzer, LED, and relay are activated.
11. The system continuously monitors the surrounding air condition.

Prototype Operation

The prototype demonstrates automatic air pollution and gas monitoring with a warning system. The MQ2 gas sensor provides the gas detection data, while the ESP8266 processes the sensor readings and controls the buzzer, LED, and relay based on the programmed condition.

Prototype Working :

- The □ ESP8266 NodeMCU is powered ON and initializes all connected components.
- The MQ2 gas sensor continuously detects smoke and combustible gases in the surrounding air.
- The sensor sends the measured gas level to the ESP8266.
- The ESP8266 processes the gas sensor reading.
- The measured gas level is compared with the predefined safety limit.
- If the gas level is above the set limit, the buzzer turns ON to alert the user.
- At the same time, the LED provides a visual warning indication.
- The relay connected to D5 (GPIO14) is activated for suitable exhaust fan or ventilation control.
- If the gas level is within the normal limit, the buzzer and LED remain OFF and the relay stays in its normal state.
- The system waits for a short interval and reads the MQ2 gas sensor again.
- This process continues automatically and continuously, providing real-time monitoring of the surrounding air condition.

11. Bill of Materials :

S.No.	Component	Specification	Quantity
1	ESP8266	NodeMCU Wi-Fi Development Board	1
2	MQ2 Gas Sensor	Smoke and Combustible Gas Sensor	1
3	Relay Module	1-Channel Relay	1
4	Buzzer	3.3V/5V Buzzer	1
5	LED	Standard Indicator LED	1
6	Resistor	Suitable LED Current Limiting Resistor	1
7	Breadboard	Standard Prototype Board	1
8	Jumper Wires	Male-Male / Male-Female	1 Set
9	USB Cable	Micro-USB	1
10	Power Supply	Suitable Regulated Supply	1

12. Results & Performance Analysis :

- The MQ2 gas sensor successfully detects smoke and combustible gases in the surrounding environment.
- The ESP8266 NodeMCU successfully receives and processes the sensor readings.
- The system provides continuous monitoring of air pollution and gas levels.
- When the gas level exceeds the predefined safety limit, the buzzer is activated.
- The LED provides a visual warning indication.
- The relay is activated to demonstrate exhaust fan or ventilation control.

- When the gas level returns to the normal range, the buzzer and LED turn OFF.
- The system provides an automatic and continuous air pollution monitoring process.

Results

- The MQ2 gas sensor successfully detects the presence of smoke and combustible gases.
- The ESP8266 NodeMCU successfully receives and processes the sensor data.
- The system continuously monitors the gas level in the surrounding environment.
- When the gas level exceeds the predefined safety limit, the buzzer turns ON.
- The LED provides a visual warning indication during an unsafe air condition.
- The relay activates to demonstrate ventilation or exhaust control.
- When the gas level returns to the normal range, the buzzer and LED turn OFF.
- The prototype demonstrates the basic operation of an Air Pollution Monitoring System.

Performance Analysis

Parameter	Expected Performance
Gas Monitoring	Continuous
Smoke Detection	Continuous
Sensor Response	Real-time Reading
High Gas Level Alert	Buzzer ON
Visual Warning	LED ON
Ventilation Control	Relay ON